HPC Project 3: Random Matrices, Maximum Eigenvalue Computation, and Tracy–Widom Distributions

Addison Olstad

April 20, 2026

Contents

1 Introduction **2**

2 Simulation **4**
2.1 Manager-Worker Architecture 4
2.2 Numerical Methods 5
2.2.1 Power Method 5
2.2.2 Schur Decomposition 6
2.3 Wall Time 7

3 Results **9**
3.1 Wall Time 9
3.2 Schur Usage 10
3.3 Distributions 10
3.4 Conclusion 15

Chapter 1

Introduction

Computing the largest eigenvalue of a single random matrix is not a particularly demanding task. The problem becomes computationally difficult, however, when one must generate and analyze thousands of large matrices. At scale, the memory requirements alone present a significant constraint: storing M matrices of size $n \times n$ in double precision requires $M \cdot n^2 \cdot 8$ bytes of RAM, which quickly exceeds the capacity of a typical personal computer. Beyond memory, there are several compounding bottlenecks, including generating random entries, computing eigenvalues, extracting the maximum, and writing results to disk.

The matrices considered here are drawn from the Gaussian Orthogonal Ensemble (GOE). Each matrix A is real and symmetric, with off-diagonal entries A_{ij} (for $i < j$) drawn independently from $\mathcal{N}(0, 1)$ and diagonal entries A_{ii} drawn from $\mathcal{N}(0, 2)$, with symmetry enforced by setting $A_{ji} = A_{ij}$. This normalization ensures that the spectral edge is located at $2\sqrt{n}$, consistent with the Tracy–Widom scaling expected to be seen at large matrix sizes [5].

To make this computation feasible, the problem is parallelized using MPI with a manager-worker layout. The task is embarrassingly parallel because each matrix realization can be processed independently of all others: the generation of a matrix, computation of its largest eigenvalue, and return of the result do not require communication between workers. The manager distributes work orders to workers, each of which performs these steps independently. The largest eigenvalue is computed using the Power Method, an iterative algorithm that efficiently approximates the dominant eigenvalue, which runs in $\mathcal{O}(n^2)$ and is sufficient for most realizations. In the case where the Power Method fails to converge, a Schur decomposition via LAPACK serves as a robust $\mathcal{O}(n^3)$ fallback, as it computes the full eigenvalue spectrum.

From the measured wall times, generating 10^3 realizations at $n = 1000$ takes about 57 seconds. Since each matrix only produces one maximum eigenvalue, 10^4 samples are needed before the tails of the distribution start to fill in properly. That corresponds to roughly 10 minutes of compute time on a single processor, assuming ideal scaling. While this isn't especially large on its own, it adds up quickly when running across multiple matrix sizes and configurations. For this reason, parallelization is still important to efficiently generate enough samples for reliable statistics.

Algorithm 1 Computation of Largest Eigenvalue per Realization

Require: Number of realizations M, matrix size n
Ensure: $\{\lambda_{\max}^{(s)}\}_{s=1}^{M}$

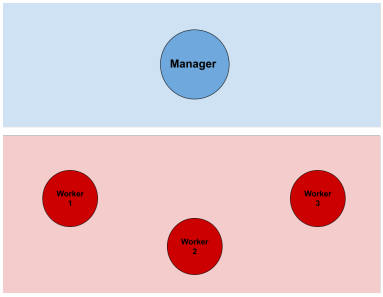
```
Initialize empty set {λ_max^(s)}_{s=1}^M
for s = 1 to M do
    Construct A^(s) ∈ ℝ^{n×n} using a PRNG
    Stage 1: Power method
    Attempt to compute dominant eigenvalue λ_max^(s) via power iteration
    if converges then
        Store λ_max^(s)
    else
        Stage 2: Shifted power method
        Apply shift μ and compute dominant eigenvalue of (A^(s) − μI)
        Recover λ_max^(s) = λ_shifted + μ if convergence achieved
        if converges then
            Store λ_max^(s)
        else
            Stage 3: Schur decomposition
            Compute full spectrum of A^(s) via Schur decomposition
            Set λ_max^(s) as largest eigenvalue
        end if
    end if
end for
return {λ_max^(s)}_{s=1}^M
```

With the ensemble of eigenvalues collected, two hypotheses are tested. The first is that the distribution of $\lambda_{\max}$ converges to a Gaussian distribution in the limit $n \to \infty$. The second is that it instead converges to the Tracy–Widom distribution predicted by random matrix theory. The Tracy–Widom distribution is the limiting distribution of the scaled largest eigenvalue of a random matrix drawn from Gaussian ensembles [6]. These hypotheses will be evaluated using goodness-of-fit tests such as the Shapiro–Wilk and Kolmogorov–Smirnov tests, along with moment-based measures (skewness and excess kurtosis), to quantify deviations from normality.

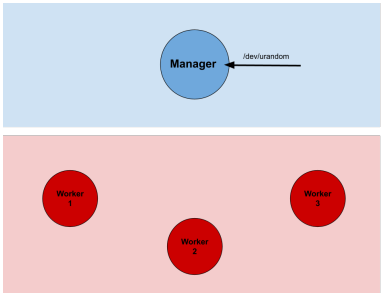
Chapter 2

Simulation

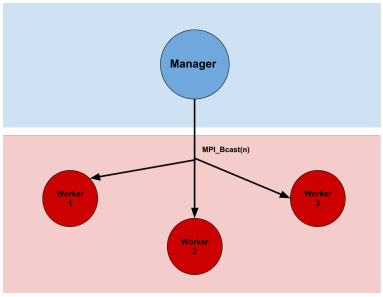
2.1 Manager-Worker Architecture



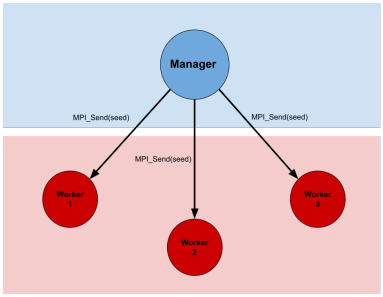
(a) Initialization



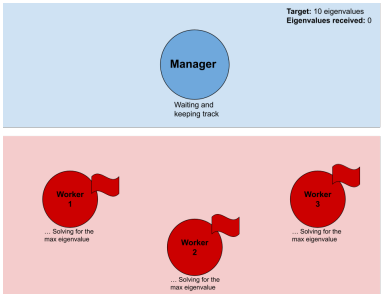
(b) Reading the Seeds



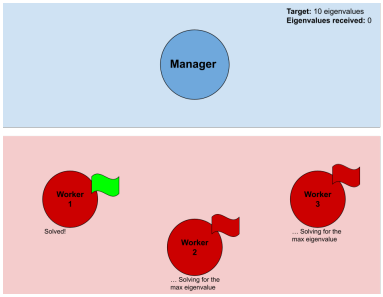
(c) Broadcasting Random Seeds



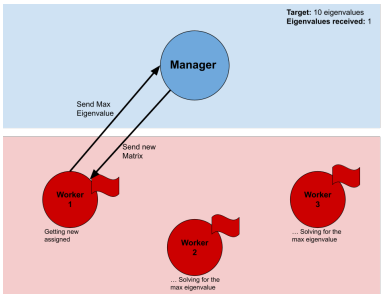
(d) Dispatching Work Orders



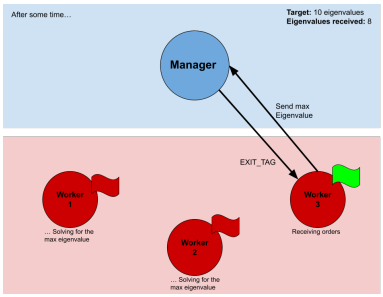
(e) Workers Computing



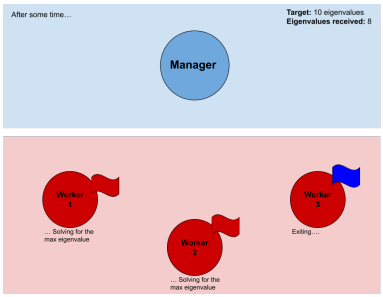
(f) First Result In



(g) Dynamic Redispatch



(h) Exit Tag Sent



(i) Winding Down

Figure 2.1: Manager–Worker MPI workflow for distributed eigenvalue computation

The computation follows a manager–worker MPI model. The manager process is responsible for

initializing the random number generator state and distributing work orders to worker processes. Each worker independently constructs a random matrix using a seed-initialized sequence, computes the largest eigenvalue using an iterative method with a deterministic fallback when necessary, and returns the result to the manager. Work is dynamically assigned as workers request new tasks, ensuring efficient utilization of available processes. Once all tasks are completed, the manager sends termination signals (exit tags) and the system shuts down.

2.2 Numerical Methods

A naive approach to computing eigenvalues is via the characteristic polynomial:

$$P_A(x) = \det(\lambda I - A) \tag{2.1}$$

For sizable matrices this is not feasible because computing it is intensive, numerically unstable, and theoretically difficult. Instead numerical iterative methods can find eigenvalues directly with higher efficiency. In this project two methods were explored within the program: the Power method (Von Mises Iteration) and the Schur decomposition.

All linear algebra operations in this work are implemented using Intel MKL BLAS and LAPACK routines. In particular, matrix–vector products are performed using the level-2 BLAS routine `dgemv`, while full eigenvalue decompositions (when required) are computed using the LAPACK routine `DGEEV`. Random number generation is handled via the MKL Vector Statistical Library (VSL).

2.2.1 Power Method

The Power method, also known as Von Mises iteration, computes the eigenvalue of a diagonalizable matrix A that is greatest in absolute value, along with its corresponding eigenvector. Each iteration costs $\mathcal{O}(n^2)$, dominated by a single matrix–vector multiplication. The method requires that A possess a *strictly dominant* eigenvalue, one that is strictly larger in absolute value than all others, and that the initial vector $\mathbf{b}_0$ have a nonzero component in the direction of the dominant eigenvector. If either condition fails, convergence is not guaranteed.

Starting from a random initial vector $\mathbf{b}_0$, each iteration multiplies by A and renormalizes:

$$\mathbf{b}_{k+1} = \frac{A\mathbf{b}_k}{\|A\mathbf{b}_k\|}.$$

The sequence $\{\mathbf{b}_k\}$ converges to the dominant eigenvector. At each step the corresponding eigenvalue is estimated via the Rayleigh quotient,

$$\lambda \approx \mathbf{b}_k^\top A\,\mathbf{b}_k,$$

and iteration continues until the relative change in λ falls below a prescribed tolerance ε:

$$\frac{|\lambda_{\text{new}} - \lambda_{\text{old}}|}{|\lambda_{\text{old}}|} < \varepsilon.$$

Eigenvalues of a large GOE matrix are distributed about zero, so the dominant eigenvalue returned by the basic algorithm may be negative. To recover the true spectral maximum, a spectral shift μ is applied: the power method is run on the shifted matrix $(A + \mu I)$, whose eigenvalues are those of A displaced by μ. On the first call $\mu = 0$; if the result is negative the method is restarted with $\mu = -\lambda$, which flips the spectrum and ensures the largest positive eigenvalue is dominant.

The spectral shift μ translates the spectrum of A so that the eigenvalue of interest becomes dominant in magnitude for the shifted matrix $(A + \mu I)$, improving convergence of the power iteration.

The subroutine `power(A, mu, eps, M, eig, conv)` accepts the matrix A, shift μ, tolerance `eps`, and maximum iteration count `M`, and returns the estimated dominant eigenvalue `eig` together with a convergence flag `conv`. Setup initializes a random starting vector drawn from $\mathcal{N}(0, 1)$ using Intel MKL's `vdrnggaussian`, which is then normalized. The value `mem`$= -100$ is used to avoid a division-by-zero in the relative-change check on the first iteration.

Algorithm 2 Power Method Subroutine

```
conv ← false
Generate Gaussian random vector v
Normalize v ← v/‖v‖
mem ← −100
for i = 1 to M do
    y ← Av
    y ← y + μv
    nrm ← ‖y‖
    eig ← v^T y
    eig ← eig − μ
    err ← |mem − eig| / |mem|
    if err < ε then
        conv ← true
        break
    end if
    v ← y/nrm
    mem ← eig
end for
```

The $\mathcal{O}(n^2)$ BLAS routine `dgemv` performs the matrix–vector product $y \leftarrow Av$ and is the computational bottleneck of each iteration. The shift is applied additively after the multiply ($y \mathrel{+}= \mu v$), and the Rayleigh quotient is corrected by subtracting the shift μ, ensuring that `eig` always represents the eigenvalue of the original matrix A rather than the shifted matrix $(A + \mu I)$ [3].

2.2.2 Schur Decomposition

When the power method fails to converge within the prescribed iteration limit, the issue is not typically a violation of the existence of a dominant eigenvalue, but rather a small spectral gap between the largest and second-largest eigenvalues. In this regime, convergence becomes extremely slow, and accumulated floating-point errors over many iterations can degrade the alignment of the iterate with the true dominant eigenvector, eventually causing stagnation. In such cases, the realization is not discarded. Instead, a fallback Schur decomposition is used to compute the full spectrum exactly.

The Schur decomposition states that any complex square matrix A is unitarily similar to an upper triangular matrix T,

$$A = QTQ^*,$$

where the diagonal entries of T are precisely the eigenvalues of A. In practice this decomposition is computed via the QR algorithm. Because it produces the full spectrum rather than a single extrema

eigenvalue, it is considerably more expensive than the power method at $\mathcal{O}(n^3)$, but it is robust and requires no assumptions about spectral dominance.

The fallback is programmed immediately after the power method returns with `conv = .false.`, using LAPACK's `DGEEV` routine to compute all eigenvalues of A:

Algorithm 3 Fallback to DGEEV

```
if ¬conv then
    DGEEV(...)
    if info = 0 then
        eig ← max(wr)
        conv ← true
    else
        print "error in DGEEV", info
    end if
end if
```

To record which method produced each result, the eigenvalue is sent to the root process via MPI with a tag encoding the convergence path: tag 0 indicates the power method converged, while tag 1 indicates the Schur fallback was used.

Algorithm 4 MPI Send Result

```
if conv then
    tag ← 0
else
    tag ← 1
end if
MPI_SEND(eig, 1, MPI_DOUBLE_PRECISION, 0, tag, MPI_COMM_WORLD, ierr)
```

These tags are collected by the root process alongside the eigenvalue data, allowing the results section to report separately on the fraction of realizations that required the fallback.

2.3 Wall Time

Wall time measures the total elapsed time from the start of program execution to completion. In this simulation, the computational cost of each realization is dominated by the eigenvalue computation. The power method requires $\mathcal{O}(n^2)$ operations per iteration, while the fallback Schur decomposition (implemented via LAPACK's `DGEEV`) requires $\mathcal{O}(n^3)$ operations. Since the fallback is only invoked when the power method fails to converge within a prescribed tolerance, the overall cost per realization lies between these two bounds and depends on the spectral properties of the sampled matrix.

All wall-time measurements are reported for ensembles of M realizations of Gaussian Orthogonal Ensemble (GOE) matrices of size $n \times n$. The power method uses a convergence tolerance of $\varepsilon = 10^{-5}$ and a maximum iteration count of $n/2$. Wall times are measured using `MPI_Wtime` and correspond to single-run executions unless explicitly averaged over multiple runs.

Under MPI parallel execution with p workers, the ensemble is distributed across processes using a

manager–worker model. In the ideal case of perfect parallel efficiency, the wall time scales as

$$T_{\text{wall}} \approx \frac{T_{\text{serial}}}{p}.$$

In practice, deviations from this scaling arise due to MPI communication overhead and intrinsic variability in per-realization compute time. In particular, convergence rates of the power method depend on the spectral gap between the largest and second-largest eigenvalues, while occasional fallback to Schur decomposition introduces significantly higher computational cost for individual realizations. This results in heterogeneous task durations across workers, which limits parallel efficiency as the number of processes increases.

This variability is most pronounced near the spectral edge of GOE matrices, where eigenvalue spacing is small and iterative convergence becomes slower. As a result, even though all workers remain continuously busy under the manager–worker scheduling scheme, the overall runtime is determined by the slowest completing tasks in the ensemble.

Chapter 3

Results

The numerical results obtained from the MPI-based GOE eigenvalue simulations are presented here, the analysis is divided into three parts: computational performance (wall time), eigenvalue computation behavior (Schur fallback usage), and statistical properties of the largest eigenvalue distribution.

3.1 Wall Time

All simulations were executed on SHARCNet using matrix sizes $n = 1000, 2000, 4000$, and 6000, with $M = 10^3$ realizations per configuration. Wall times were measured using `MPI_Wtime` and correspond to single-run executions.

The observed scaling is compared to the theoretical bounds of the eigenvalue computation: $\mathcal{O}(n^2)$ for the power method and $\mathcal{O}(n^3)$ for the Schur decomposition fallback. The resulting wall time is therefore expected to lie between these regimes depending on convergence behavior.

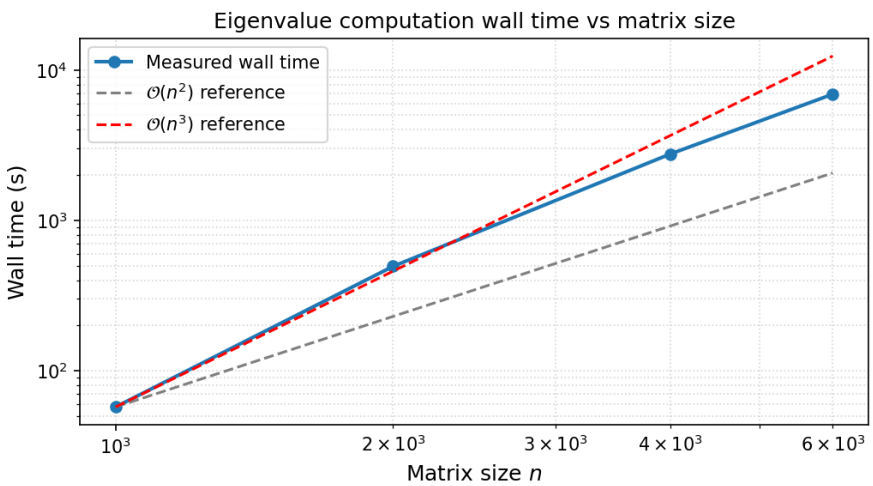


Figure 3.1: Wall time as a function of matrix size n, computed on SHARCNet for $M = 10^3$ realizations per n.

The results correspond to the hypothesis, except at $n = 2000$, when the wall time briefly exceeded the $\mathcal{O}(n^3)$ upper bound.

3.2 Schur Usage

To monitor numerical stability of the iterative solver, each eigenvalue computation was tagged according to whether it converged via the power method or required fallback to Schur decomposition. These tags were collected by the MPI root process and used to compute the fallback rate.

Quantity	**Value**
Total eigenvalues collected	1000
Power method converged	1000
Schur fallback used	0
Schur fallback rate	0.00%

Table 3.1: Eigenvalue computation summary for $n = 6000$, with identical behavior observed for all tested matrix sizes.

The absence of fallback usage suggests that, for the GOE matrices of the sizes considered, the spectral gap is typically sufficient for reliable convergence of the power method within the prescribed tolerance $\varepsilon = 10^{-5}$, and Schur decomposition never had to be utilized.

3.3 Distributions

The following figures show the empirical distribution of the largest eigenvalue $\lambda_{\max}$ for different matrix sizes, and will contain tables statistical information in accordance to the histograms.

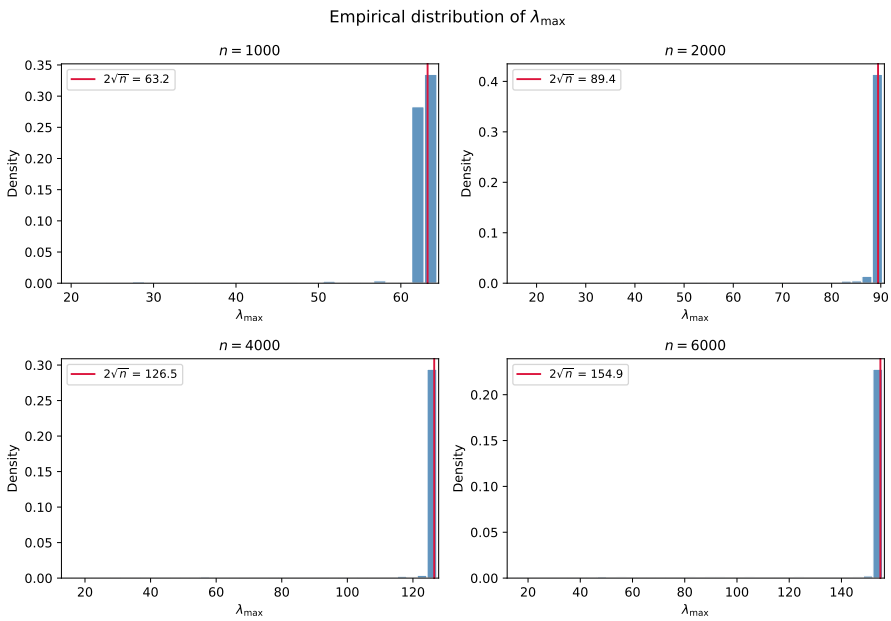


Figure 3.2: Raw SHARCnet Data of λ_{max} for $n = 1000, 2000, 4000, 6000$

The raw distributions show λ_{max} in its natural units: the actual eigenvalue values. Each panel has a different x-axis (n=1000 goes up to 65, n=6000 goes up to 155) because the spectral edge $2\sqrt{n}$. $2\sqrt{n}$ shifts the martrix size. The red line marks $2\sqrt{n}$ and it can be seen how the bulk of the eigenvalues cluster just next to it. The distribution is very tight and hard to see, so it just looks like a spike near the edge.

n	Total Samples	In Window [-2,2]	Mean (z)	Std (z)
1000	1000	961	-0.455	1.853
2000	1000	918	-1.063	3.583
4000	1000	903	-1.477	4.858
6000	1000	873	-2.429	6.846

Table 3.2: Summary of Tracy–Widom normalized eigenvalues from the Narval data with $n_{\text{dat}} = 1000$. The "In Window" column indicates how many samples fall within the plotted range $[-2, 2]$, but all eigenvalues were used in the Mean and Std calculations.

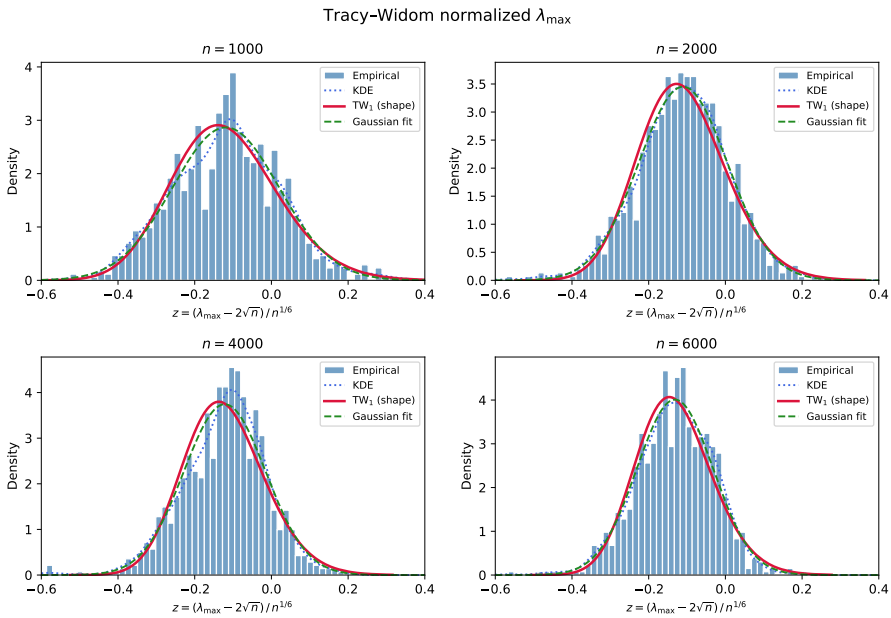


Figure 3.3: Normalized SHARCnet Data of λ_{max} for $n = 1000, 2000, 4000, 6000$

To normalize the distribution to the Tracy–Widom behavior, the appropriate scaling is applied to the eigenvalues:

$$z = \frac{\lambda_{\max} - 2\sqrt{n}}{n^{1/6}} \tag{3.1}$$

By subtracting $2\sqrt{n}$ it centers all the distributions at zero regardless of n. Dividing by $n^{1/6}$ rescales the fluctuations. This is the key prediction of Tracy–Widom theory.

After this transformation, if Tracy–Widom theory is correct, all four panels should exhibit the same limiting shape, converging to the TW_1 distribution as n increases. The TW_1 curve (red) is shown rescaled to match the empirical mean and standard deviation, so the comparison emphasizes the *shape* rather than the scale.

The empirical distribution appears slightly narrower than TW_1 due to finite-n effects in the GOE normalization, as this run only used $ndat = 1000$. However, the characteristic right-skewed and asymmetric profile is qualitatively consistent with TW_1. The right tail deviates from the symmetric Gaussian fit (green dashed), though the limited data makes a definitive conclusion difficult.

n	SW p	$\mathrm{KS}_{\mathcal{N}}$ D	$\mathrm{KS}_{\mathcal{N}}$ p	$\mathrm{KS}_{\mathrm{TW}}$ D	Skewness	Ex. Kurtosis
1000	4.22e-53	0.4523	2.48e-187	0.6884	-6.5706	47.0570
2000	3.01e-51	0.4631	6.92e-197	0.6405	-4.2757	18.3641
4000	4.92e-51	0.4707	9.88e-204	0.6330	-4.0104	15.8639
6000	2.25e-49	0.4689	3.62e-202	0.6064	-3.1474	9.0309

Table 3.3: Goodness-of-fit statistics for $n_{\mathrm{dat}} = 1000$. Small Shapiro–Wilk p-values reject normality.

To assess the distributional behavior of the largest eigenvalues, several standard goodness-of-fit diagnostics were put against both a Gaussian reference distribution and the Tracy–Widom law.

The Shapiro–Wilk (SW) test evaluates the null hypothesis that the sample is normally distributed. Its p-value quantifies evidence against normality: $p < 0.05$ indicates a strong rejection of the Gaussian hypothesis. In all cases here, all the p-values are extremely small, providing evidence against normality [4].

The Kolmogorov–Smirnov (KS) statistic, D, measures the maximum distance between the empirical cumulative distribution function and the reference CDF. Larger values of D indicate poorer agreement with the proposed model. The KS distances are computed both against a standard normal distribution ($\mathrm{KS}_{\mathcal{N}}$) and against the Tracy–Widom distribution ($\mathrm{KS}_{\mathrm{TW}}$), allowing direct comparison of which model better describes the data [1]. So a value near $D = 0$ would mean the distributions are alike, a value near $D = 0.2$ would indicate the distribution deviate, and a value near $D = 0.5$ indicate the distributions are heavily different. The values in this table near $D = 0.4$, providing evidence against normality.

Skewness measures the asymmetry of the distribution, with zero corresponding to perfect symmetry. Excess kurtosis measures tail heaviness relative to the normal distribution, where positive values indicate heavier tails. The strongly negative skewness and large positive excess kurtosis across all n indicate substantial deviation from Gaussian behaviour, consistent with the heavy-tailed structure expected in random matrix eigenvalue fluctuations [2].

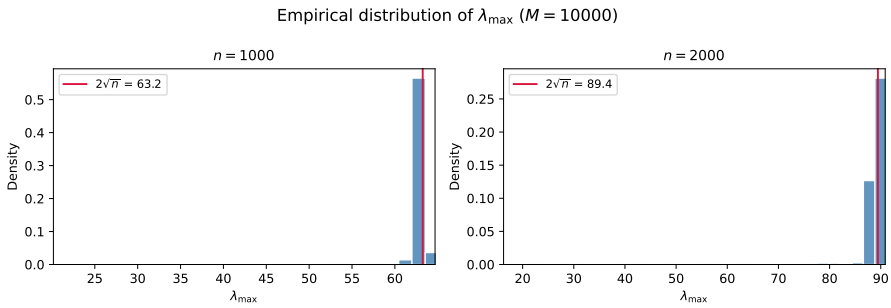


Figure 3.4: Raw Data at $n = 1000, 2000$ at $M = 10000$

Raw data is provided for larger values of $ndat$, so the tails of the distribution could hypothetically fill up more, allowing for more confident statistically analysis.

n	Total Samples	In Window [-2,2]	Mean (z)	Std (z)
1000	10000	9603	-0.434	1.755
2000	10000	9277	-0.840	3.041

Table 3.4: Summary of Tracy–Widom normalized eigenvalues for Narval data with $n_{\text{dat}} = 10000$.

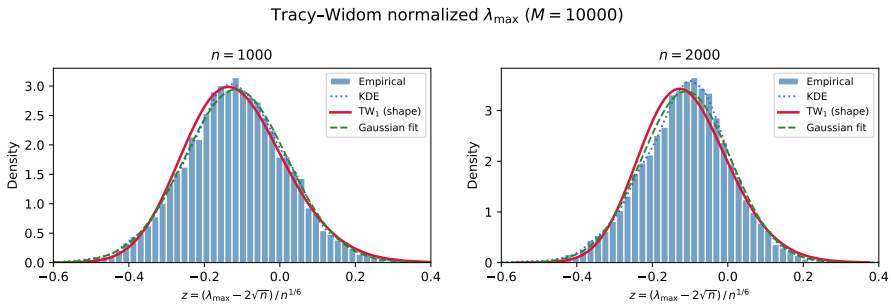


Figure 3.5: Normalized data at $n = 1000, 2000$ at $M = 10000$

These histograms, normalized for Tracy–Widom do show a well filled out distribution, with tails consisiting of more data. the distributions can be shown overlayed on the data.

n	SW p	$\text{KS}_{\mathcal{N}}$ D	$\text{KS}_{\mathcal{N}}$ p	KS_{TW} D	Skewness	Ex. Kurtosis
1000	1.95e-89	0.4414	0.00e+00	0.6817	-6.6298	47.9670
2000	7.88e-88	0.4647	0.00e+00	0.6560	-4.8815	24.8670

Table 3.5: Goodness-of-fit statistics for $n_{\text{dat}} = 10000$.

The Shapiro–Wilk test, along with the Kolmogorov–Smirnov (KS) statistics, and skewness and Kurtosis are provided for the larger distribution. The results indicate non-normality. The skewness indicates a strong left tail, as shown in the raw data. The KS test both rejects normality and the Tracy–Widom distribution as good fits, most likely because of this tail.

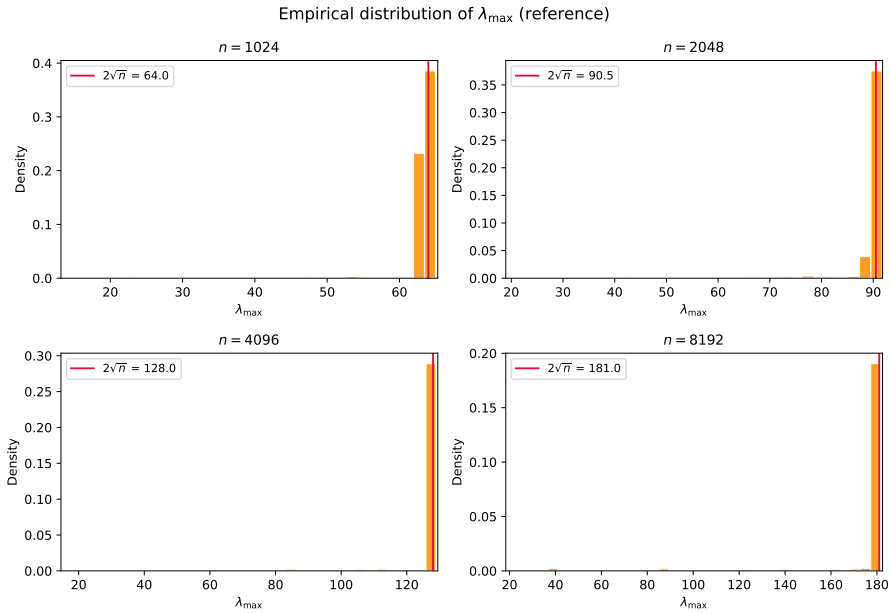


Figure 3.6: Raw Data from Connor's Dataset

n	Total Samples	In Window [-2,2]	Mean (z)	Std (z)
1024	1024	980	-0.498	2.068
2048	1024	948	-0.817	2.973
4096	1024	904	-1.742	5.259
8192	1024	869	-3.049	7.960

Table 3.6: Summary of Tracy–Widom normalized eigenvalues for reference data.

N values are shown with a consistent sample size, alongside the Mean and Std.

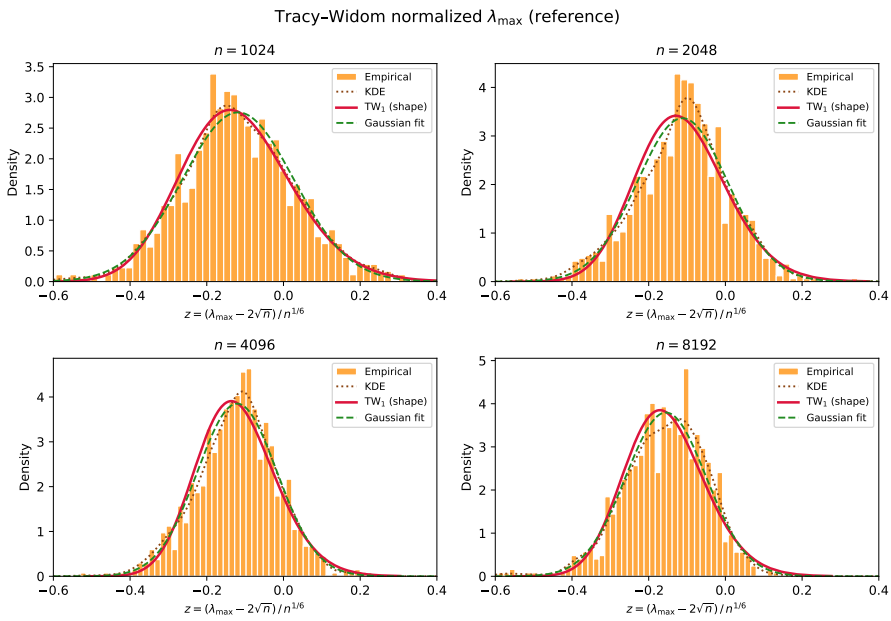


Figure 3.7: Tracy–Widom normalized histograms

The plots are shown with a focused *xlim* around the main maximum. The distributions are laid overtop the empirical data.

n	SW p	$\mathrm{KS}_{\mathcal{N}}$ D	$\mathrm{KS}_{\mathcal{N}}$ p	$\mathrm{KS}_{\mathrm{TW}}$ D	Skewness	Ex. Kurtosis
1024	1.28e-53	0.4508	1.76e-190	0.6794	-6.2773	41.1222
2048	1.97e-52	0.4615	4.19e-200	0.6534	-5.0597	27.0296
4096	1.47e-50	0.4750	1.02e-212	0.6250	-3.5639	12.2011
8192	6.29e-49	0.4544	1.06e-193	0.5706	-2.8661	7.1742

Table 3.7: Goodness-of-fit statistics for reference dataset.

The goodness-of-fit tests indicate the same as they did for the previous results, showing a confident deviation from a normal distribution, with strong left tails, but at a $KS_{TW}D = 0.6$ average, the tests also indicate a deviation away from the Tracy Widom distribution when testing with the tails included.

3.4 Conclusion

This project investigated the computation of the largest eigenvalue of Gaussian Orthogonal Ensemble (GOE) matrices using a parallel MPI implementation, with the goal of both improving computational efficiency and examining the statistical behavior of the resulting eigenvalue distributions.

From a computational perspective, the manager–worker model proved well-suited to this problem due to its embarrassingly parallel structure. Each realization can be processed independently, allowing near-ideal workload distribution across workers. The observed wall time scaling is consistent with theoretical expectations: the dominant cost per realization lies between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$, depending on convergence behavior.

The power method was found to be highly reliable for the matrix sizes considered. No instances of non-convergence were observed, and therefore the Schur decomposition fallback was not required in practice. Importantly, the inclusion of a deterministic fallback ensures that, even in cases of slow convergence, eigenvalues are still computed rather than discarded. This guarantees that the ensemble remains unbiased, as all realizations contribute to the final distribution.

From a statistical perspective, the results show a clear and consistent deviation from Gaussian behavior. Across all matrix sizes and ensemble sizes tested, the Shapiro–Wilk test strongly rejects normality, and Kolmogorov–Smirnov statistics indicate poor agreement with a Gaussian model. This deviation is further supported by large negative skewness and significant excess kurtosis, reflecting the asymmetric and heavy-tailed nature of the distribution.

After applying the appropriate scaling, the empirical distributions exhibit qualitative agreement with the Tracy–Widom TW_1 distribution. In particular, the characteristic asymmetry and tail behavior become visible, especially as the ensemble size increases to $M = 10^4$. Although the goodness-of-fit tests still report large deviations—possibly due to finite-size effects and limited sampling in the tails.

In summary, the results support the conclusion that the largest eigenvalue of GOE matrices does not converge to a Gaussian distribution, but instead qualitatively follow the Tracy–Widom law in the large-n limit. Parallelization via MPI was essential in making these large-scale simulations computationally feasible, enabling sufficient sampling to observe this non-Gaussian behavior.

Bibliography

[1] National Institute of Standards and Technology. Kolmogorov–smirnov goodness-of-fit test (eda section 3.5.3). https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm, 2026. Accessed: 2026-04-20.

[2] National Institute of Standards and Technology. Shapiro–wilk test for normality (eda section 3.5.7). https://www.itl.nist.gov/div898/handbook/eda/section3/eda35g.htm, 2026. Accessed: 2026-04-20.

[3] Wikipedia contributors. Rayleigh quotient. https://en.wikipedia.org/wiki/Rayleigh_quotient, 2026. Accessed: 2026-04-20.

[4] Wikipedia contributors. Shapiro–wilk test. https://en.wikipedia.org/wiki/Shapiro%E2%80%93Wilk_test, 2026. Accessed: 2026-04-20.

[5] Wikipedia contributors. Tracy–widom distribution. https://en.wikipedia.org/wiki/Tracy%E2%80%93Widom_distribution, 2026. Accessed: 2026-04-17.

[6] Wolfram Research. Tracy–widom distribution. https://www.wolfram.com/language/11/random-matrices/tracy-widom-distribution.html, 2015. Accessed: 2026-04-15.